



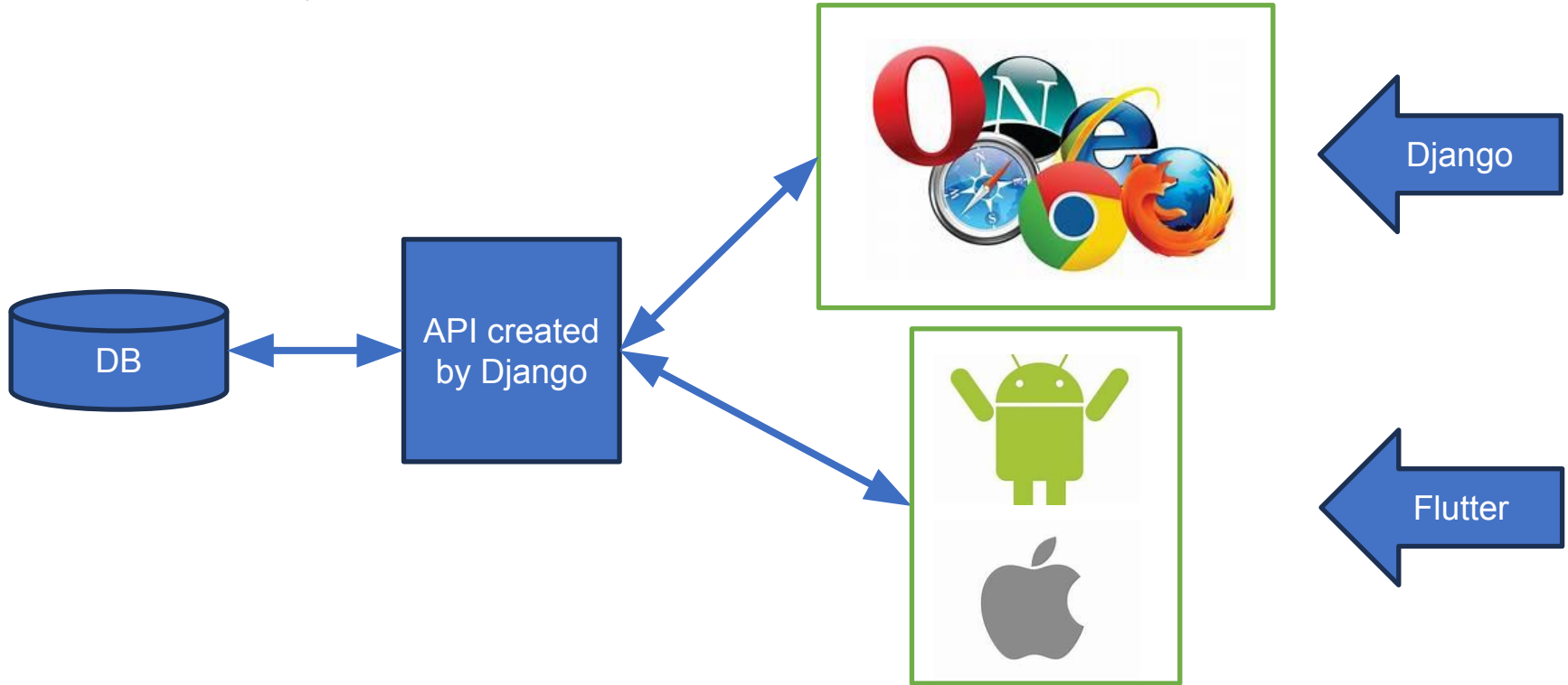
Introduction to the Internet and Web Framework

Tim Dosen PBP

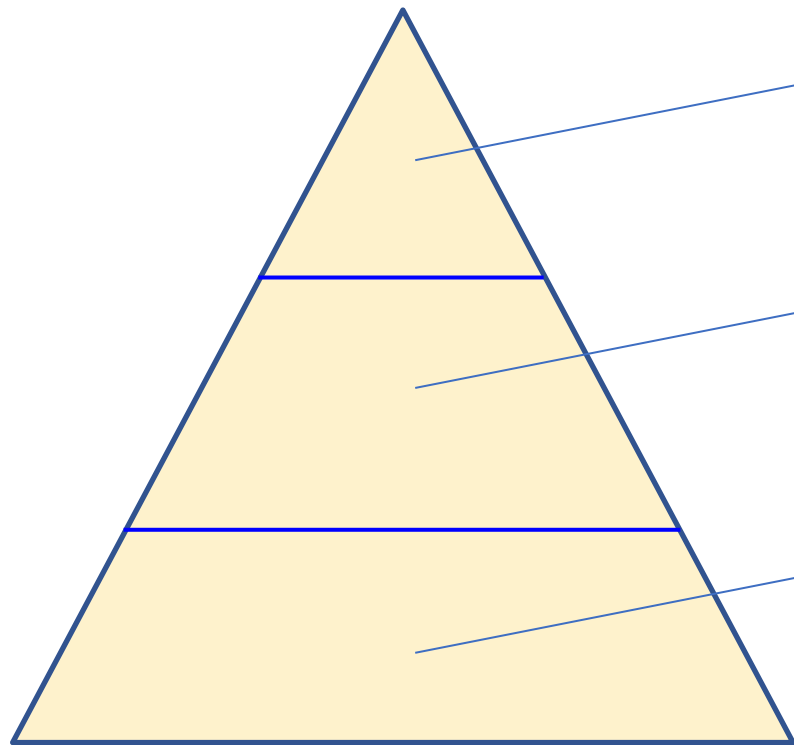
Previous Session Topics

- Basic Concepts of Platforms
- Programming Frameworks
- Web Platforms vs. Mobile Platforms

In General, we can do:



Forms of Cloud Computing



Software as a Service (SaaS)

Ready-to-use application software running on Cloud infrastructures. Examples: Gmail, Microsoft Office 365, and Dropbox.

Platform as a Service (PaaS)

It offers full or partial application development software and libraries for developers such as Google App Engine and AWS Elastic Beanstalk which are callable over the internet.

Infrastructure as a Service (IaaS)

It delivers raw computer infrastructure in the form of virtual machines which can be scaled up and down based on application resource needs. Examples: Amazon Elastic Compute Cloud (Amazon EC2), Google Compute Engine (GCE), and IBM Cloud.

CI/CD

- **Continuous integration (CI)** automatically **builds, tests, and integrates** code changes within a shared repository
- **Continuous delivery (CD)** automatically **delivers** code changes to production-ready environments for approval
- **Continuous deployment (CD)** automatically **deploys** code changes to customers directly

Agenda

- Basic concepts of the Internet
- How webserver works
- Web framework

Introduction to the Internet

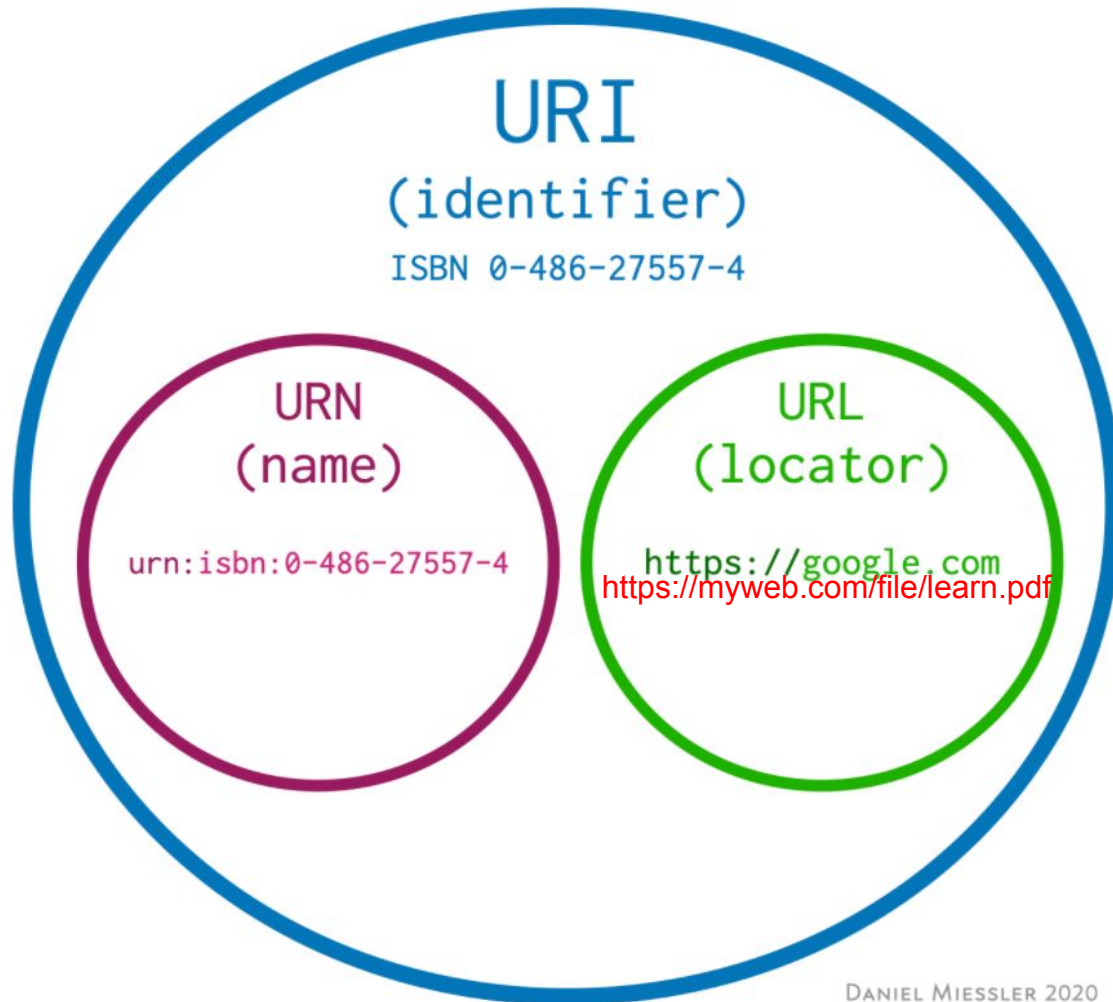
The Internet & the Web

- The Internet: a global network of computers

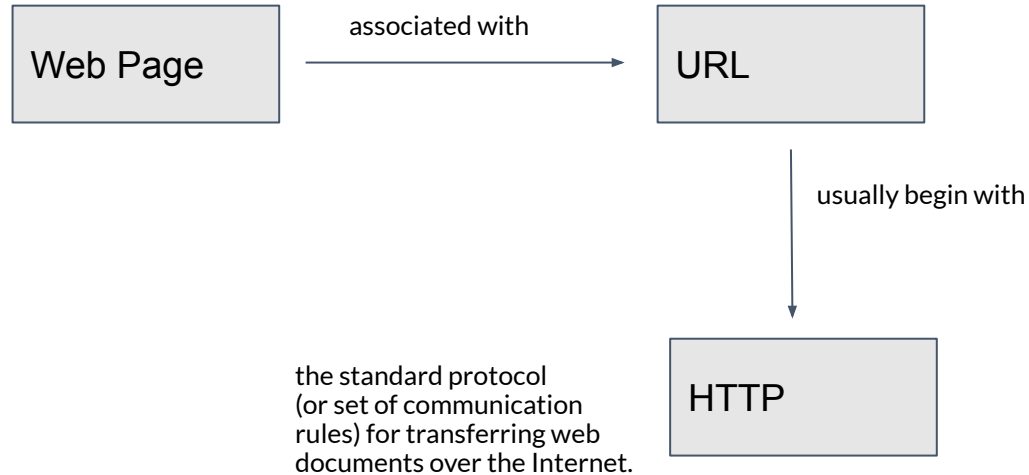


The World Wide Web (WWW, or simply Web) is an **information space** in which the items of interest, referred to as resources, are identified by global identifiers called Uniform Resource Identifiers (URI).

-(W3C's [Architecture of the World Wide Web, Volume I](#))



URL



The URL specifies the address (i.e., location) of the web page displayed in the browser window.

URL Syntax

- Every HTTP URL conforms to the syntax of a generic URI.
- The generic URI syntax consists of a hierarchical sequence of components referred to as the scheme, authority, path, query, and fragment.

URI = scheme ":" hier-part ["?" query] ["#" fragment]

hier-part = "/" authority path-abempty begins with "/" or is empty
/ path-absolute
/ path-rootless
/ path-empty

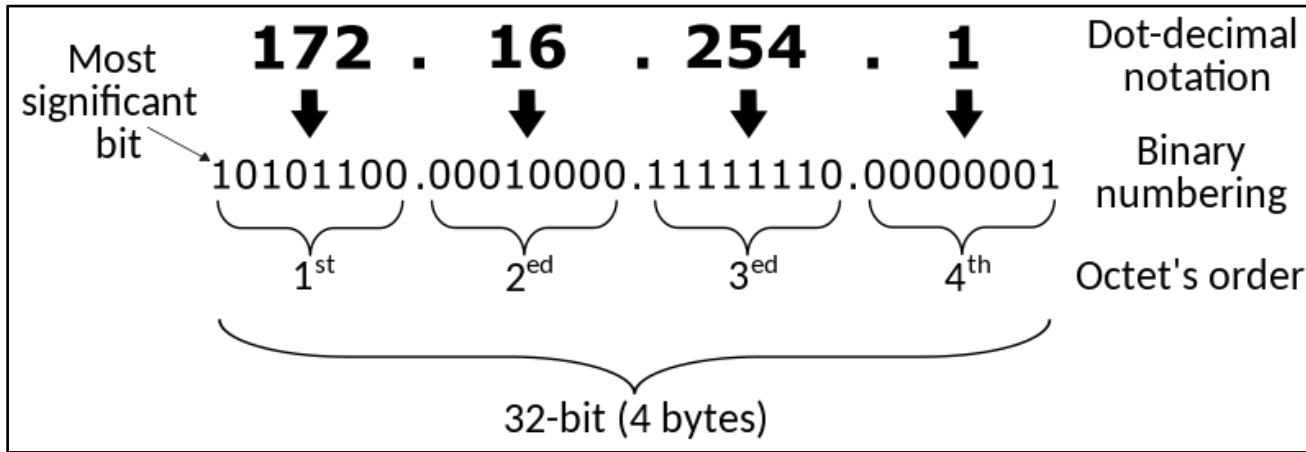
- Example: 

URL Syntax

- Example:  `http://localhost:8000/contact/person?name=pakbepe#phone`
- Examples of popular schemes include **http** (hypertext transfer protocol), **https** (hypertext transfer protocol secure), **ftp** (file transfer protocol), and **mailto**.
- The host subcomponent of authority is identified by an IP literal encapsulated within square brackets (IPv6 or IPvFuture), an IPv4 address in dotted-decimal form, or a registered name (hostname).
 - `http://localhost:8000/` → **hostname**
 - `http://127.0.0.1:8000/` → **IPv4 address**
- Hostname is translated into IP (Internet Protocol) address by DNS (Domain Name System).
- IP address determines the location of the server.
- Port number is associated with service or program on the server.

URL – IPv4 Address

- IPv4 addresses are usually represented in dot-decimal notation, consisting of four decimal numbers, each ranging from 0 to 255, separated by dots.
- Each part represents a group of 8 bits (an octet) of the address.

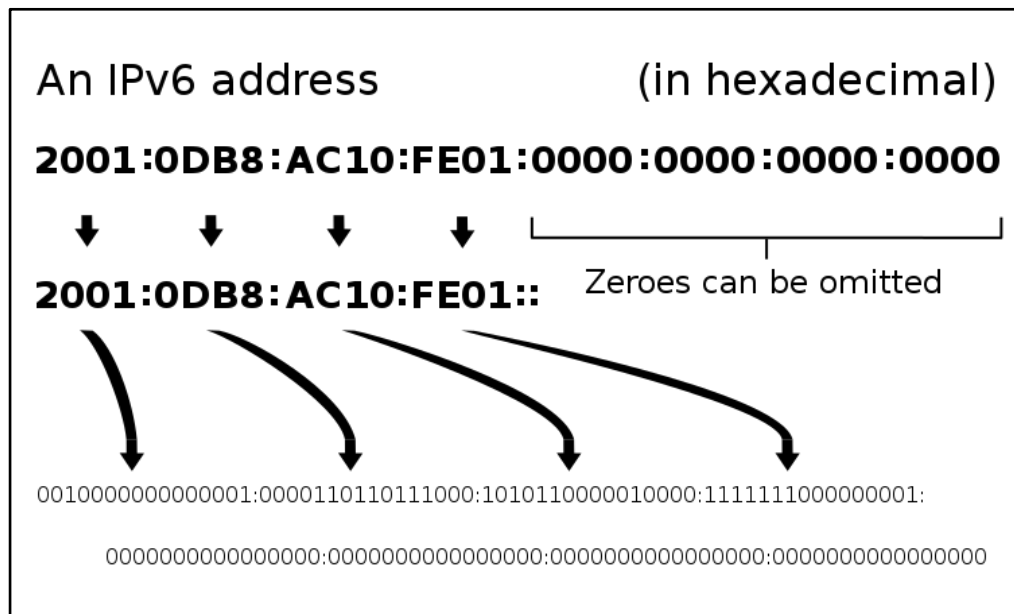


Picture from:

https://upload.wikimedia.org/wikipedia/commons/thumb/6/66/IPv4_address_structure_and_writing_systems-en.svg/799px-IPv4_address_structure_and_writing_systems-en.svg.png

URL – IPv6 Address

- In IPv6, the address size was increased from 32 bits in IPv4 to 128 bits, thus providing up to 2^{128} addresses.



URL Examples

1. <https://www.example.com/>
2. <https://www.example.com/index.html>
3. <https://www.example.com/index.php>
4. <https://example.com/>

<https://example.com/#contact>

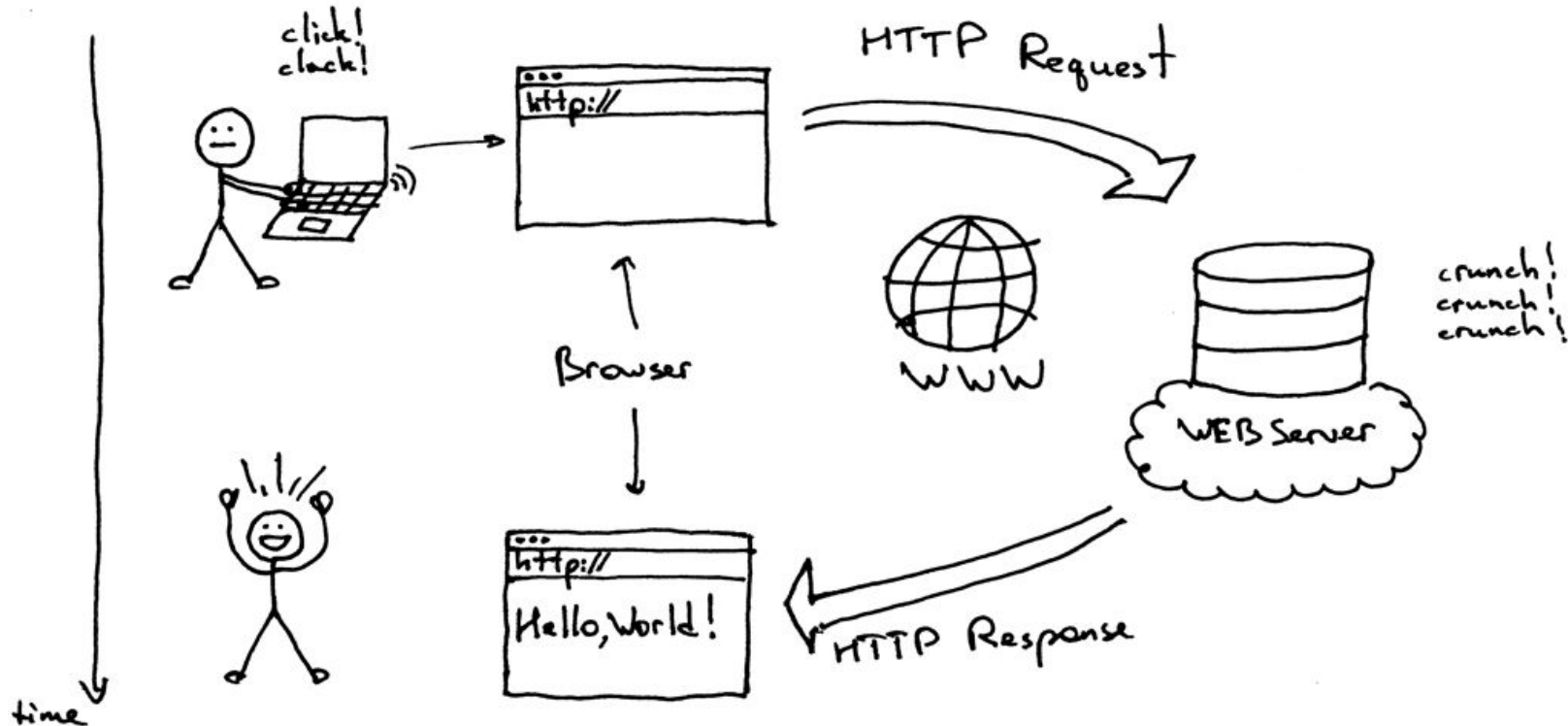
Which one
to choose?

1. <https://example.com/username/>
2. <https://example.com/username/gallery/>
3. <https://username.example.com/>
4. <https://username.example.com/gallery/>

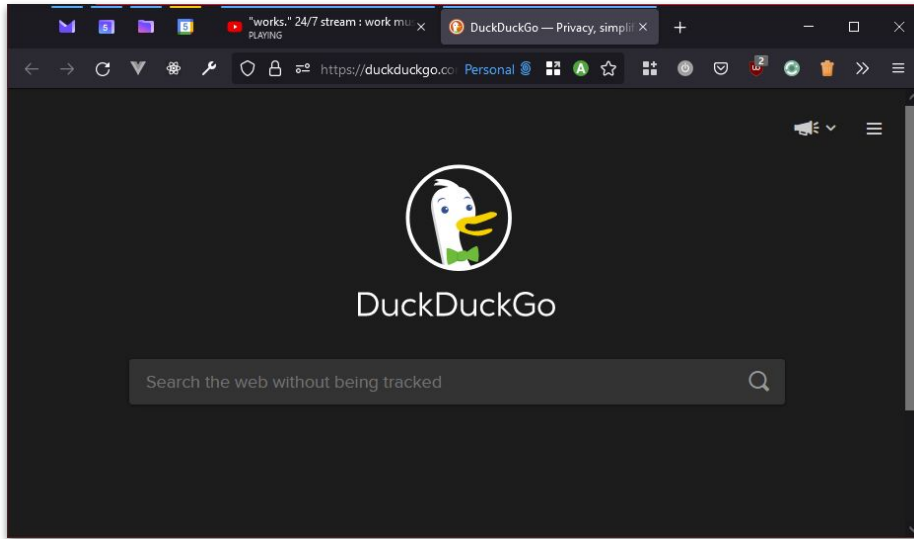
1. <https://en.example.com/>
2. <https://example.com/en/>

1. <https://www.youtube.com/watch?v=PH2QZHv4Ocw>
2. <https://youtu.be/PH2QZHv4Ocw>
3. <https://www.youtube.com/watch?v=PH2QZHv4Ocw&feature=youtu.be>
4. https://www.youtube.com/playlist?list=PLxuSBzNBcQcqLyLA_6ZaY1Rlr0QgdpPOg
5. https://www.youtube.com/watch?v=PH2QZHv4Ocw&list=PLxuSBzNBcQcqLyLA_6ZaY1Rlr0QgdpPOg&index=2

Web Request / Response



Example: Search Engine



<https://duckduckgo.com>

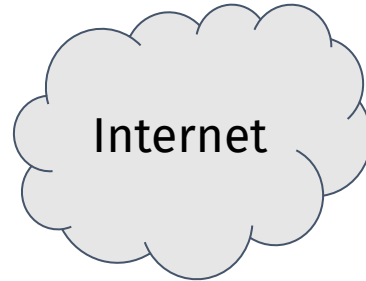
- How do you open your preferred search engine via Web browser?
- Once you clicked “Enter” key, do you know what happened in the process between your Web browser and the search engine?

Example: Search Engine (*really simplified*)



<https://duckduckgo.com>

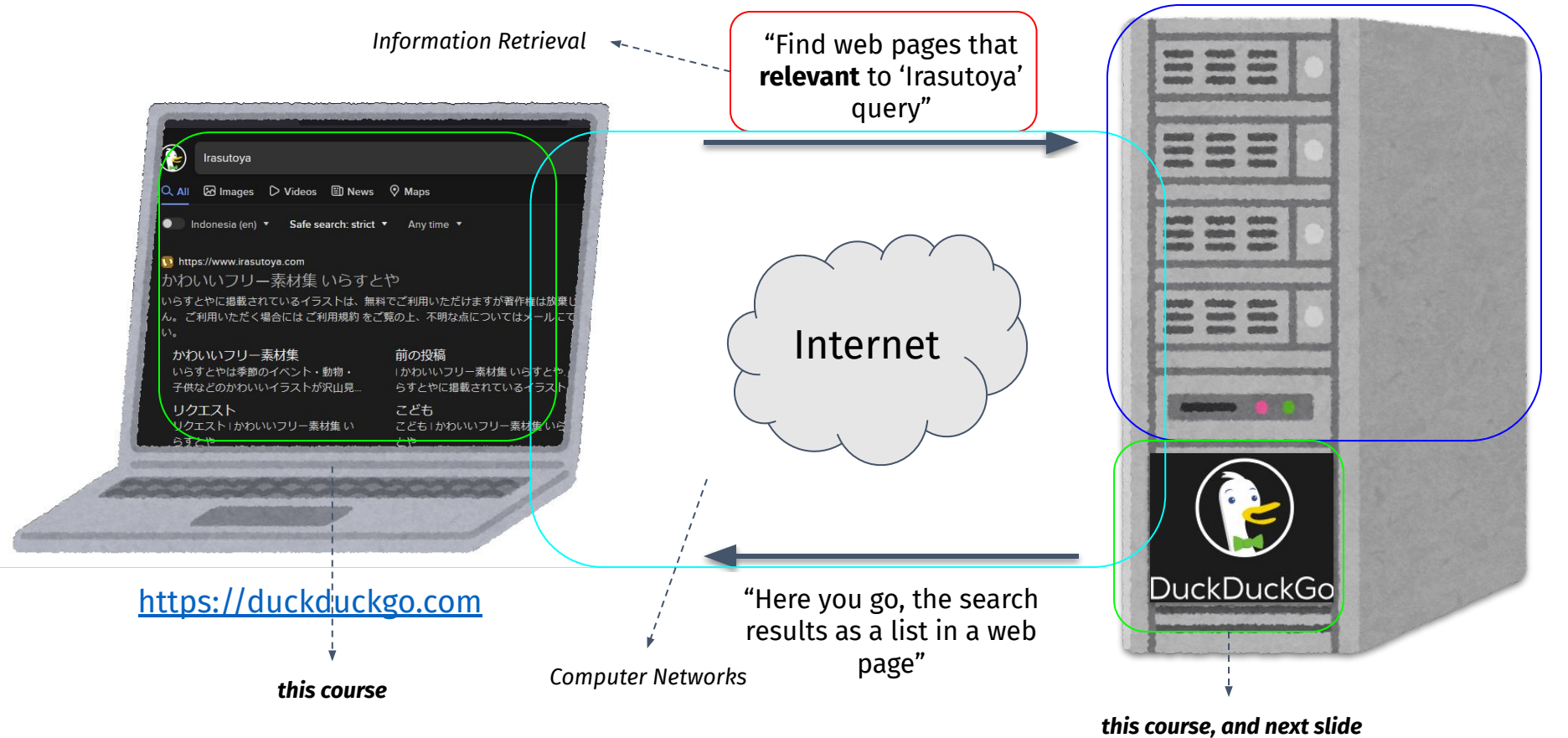
"Get the landing page from
<https://duckduckgo.com>"



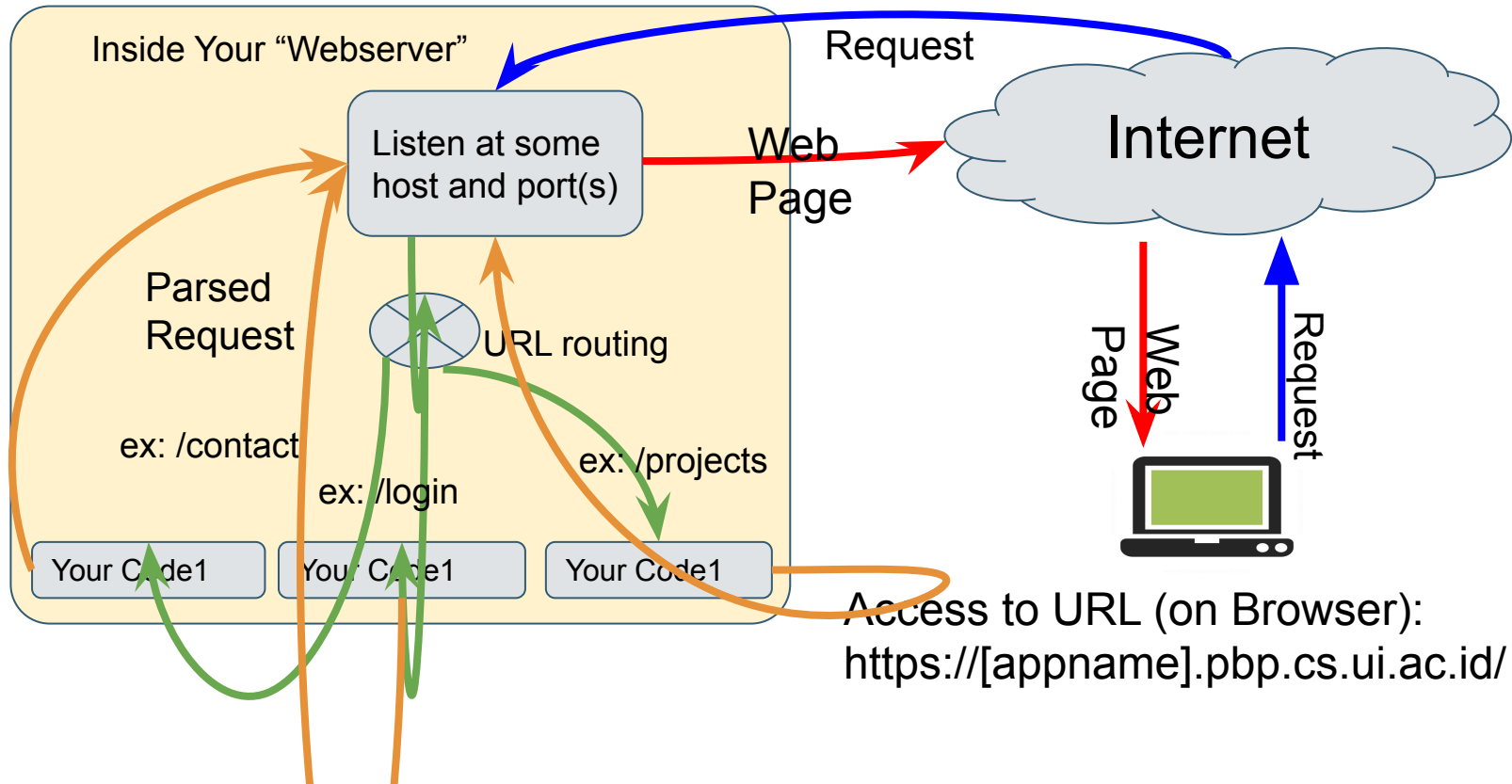
"Here you go, a web page"



Example: Search Engine (*really simplified*)



How Webserver Works



Web Programming

Web Programming

The Internet and
web programming technologies

To design web pages and applications
that run across an enormous range of
Internet-enabled devices.

- **Client-side programming** technologies are used to build web pages and applications that are run on the client (i.e., in the browser on the user's device).
- **Server-side programming**—the applications that respond to requests from client-side web browsers, such as searching the Internet, checking your bank-account balance, ordering a book from Amazon, bidding on an eBay auction and ordering concert tickets.

Client vs Server Side Scripting

CLIENT SIDE SCRIPTING	SERVER SIDE SCRIPTING
- Runs on the user's computer (Browser or scripting host)	- Executed on the web server
- Depends on browser → Browser must support the scripting language	- Not depend on client: Wide Range Programmatic capabilities→ Programmers have more flexibility
- User can view the script by using the browser's source viewing capability	- User cannot view the source code because the response is an HTML Page

Server Side Scripting Language



Client Side Scripting Languages

HTML



CSS



JS



Review

A programming framework is

a **prepackaged set of solutions** that solves common development problems.

Web Framework

Web Framework

- A web framework is a set of libraries and useful tools that can be used to build dynamic web applications.
- A web application framework forms generic web-based software application that is extended and modified by developers to create a specific application.

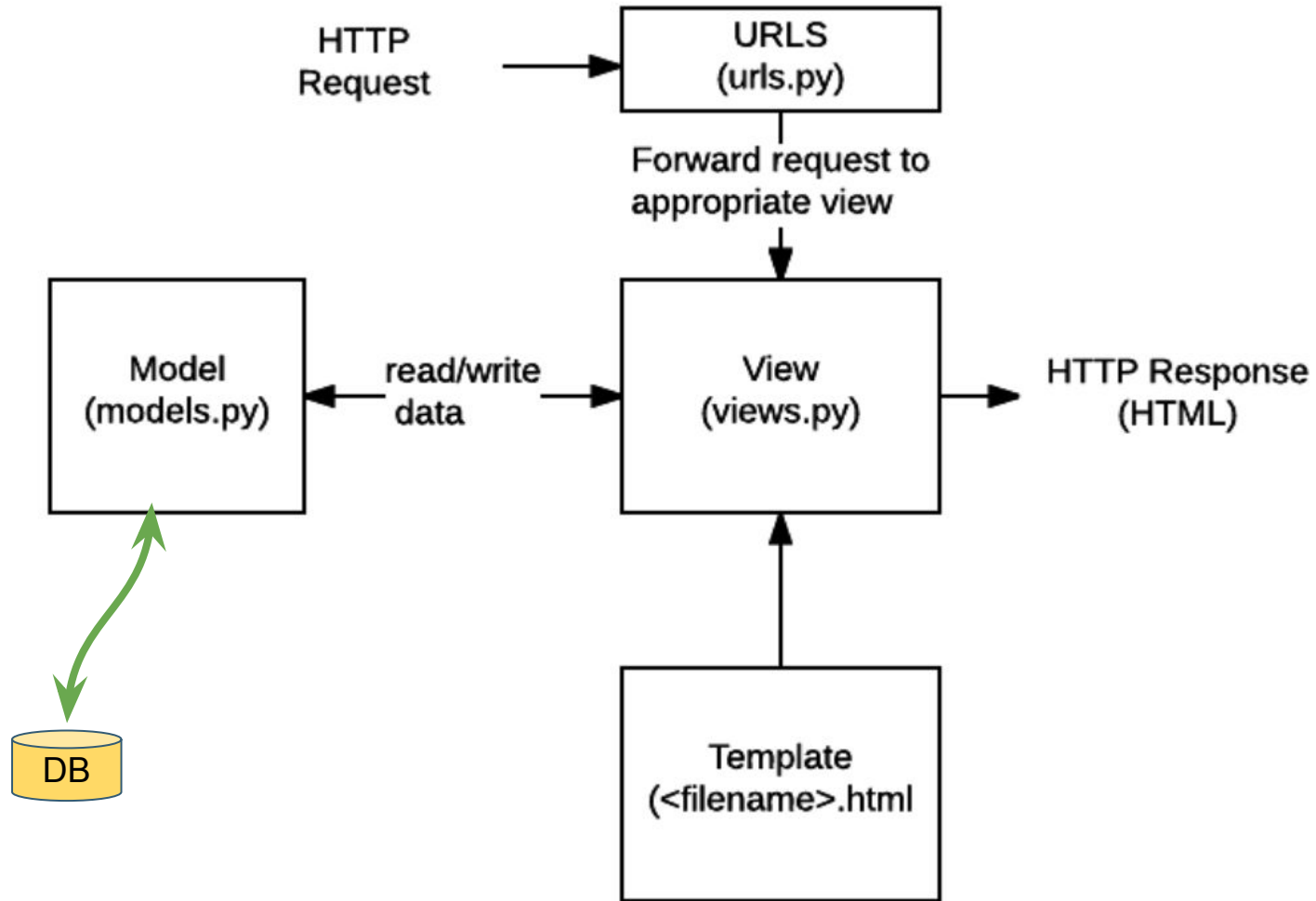
Why we use web frameworks

- Efficiency: provide basic building blocks
- Code reusing
- Separation of concerns
- Community support
- Structure matters

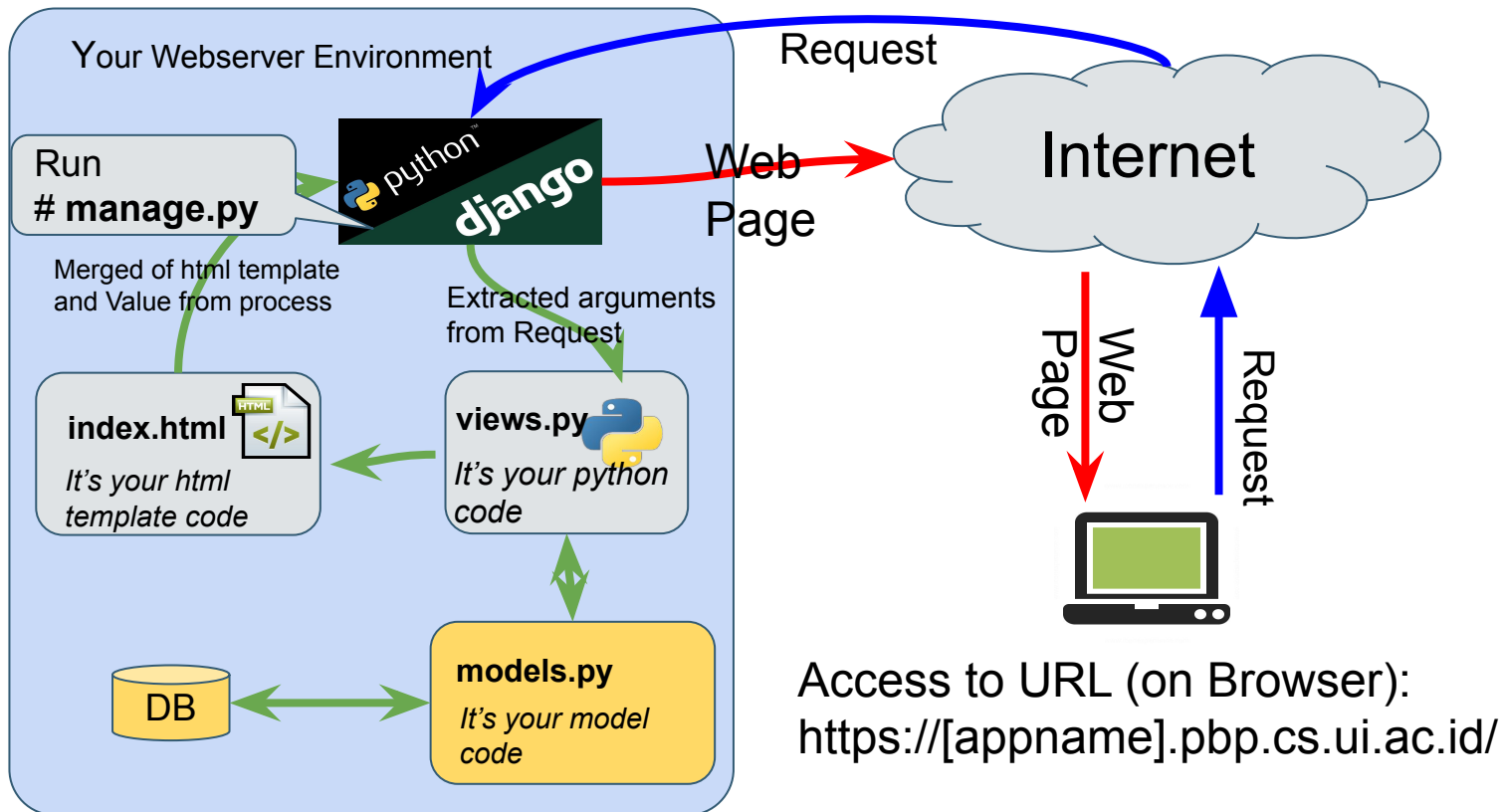
Get Stuff Done!

Django

- <https://www.djangoproject.com/>
- Language programming: Python
- Architectural pattern: Model–Template–Views (MTV)
- Why we use Django?
 - ❑ Open source
 - ❑ Ridiculously fast
 - ❑ Fully loaded
 - ❑ Reassuringly secure
 - ❑ Exceedingly scalable
 - ❑ Incredibly versatile



How Django Framework Works



Other Frameworks

Spring

- <https://spring.io/projects/spring-boot>
- Language programming: Java
- Architectural pattern: Model-View-Controller (MVC)

Laravel

- <https://laravel.com/>
- Language programming: PHP
- Architectural pattern: Model-View-Controller (MVC)

Yii

- <https://www.yiiframework.com/>
- Language programming: PHP
- Architectural pattern: Model-View-Controller (MVC)

//Choosing tools, any thoughts?

Common Features of Web Framework

- url routing
- views
- database interaction
- form controls
- user management
- session management

A programming framework is a **prepackaged set of solutions** that solves common development problems.

//what else?

Summary

In this session, students have learned about:

- The basic concepts of the Internet
- How webserver works
- Web framework

Next Session Topics

- Django architecture
- TDD (Test Driven Development)

References

- Deitel: Internet & World Wide Web How to Program 4th Edition, Prentice Hall, 2008.
- Deitel: Internet & World Wide Web How to Program 5th Edition, Prentice Hall, 2011.
- "What is the difference between the Web and the Internet?", accessed on <https://www.w3.org/Help/#webinternet> (July 2024).
- "Syntax Components", accessed on <https://datatracker.ietf.org/doc/html/rfc3986#section-3> (July 2024).